Unit - 2

Question:
Explain how classes are created in C++ and describe the role of access specifiers.

Answer:

- Classes are created using the keyword `class`.
- A class declaration defines a new type that links code and data.
- This new type is then used to declare objects of that class.
- A class is a logical abstraction, whereas an object has physical existence.
- An object is an instance of a class.

Access Specifiers in C++:

- The three access specifiers in C++ are:
  - `public`
  - `private`
  - `protected`
- By default, functions and data declared within a class are `private`.
- `private` members can be accessed only by other members of the same class. Private members and methods are for internal use only.
- The `public` specifier allows members to be accessible to other parts of the program. The public stuff is the interface.
- The `protected` specifier is used when inheritance is involved.
- Once an access specifier is used, it remains in effect until another specifier is encountered or until the class declaration ends.
- Access specifications can be changed multiple times within a class declaration.

Question:
What are member functions and member variables in a class? Mention restrictions on class members and illustrate access to public variables with an example.

Answer:

- Functions declared within a class are called member functions.
- Member functions may access any element of their class, including private elements.
- Variables that are elements of a class are called member variables or data members.
- The term instance variable is also used.
- Collectively, any element of a class is called a member of that class.

<u>Restrictions on Class Members:</u>

- A non-static member variable cannot have an initializer.
- No member can be an object of the class being declared.
- A member can be a pointer to the class being declared.
- No member can be declared as auto, extern, or register.

Accessing Public Variables:

- When a variable is declared `public`, it can be accessed directly by any part of the program.
- Syntax: Use the object name, dot operator, and the variable name.

Example:

```
#include <iostream>
using namespace std;

class myclass {
public:
    int i, j, k; // accessible to entire program
};

int main() {
    myclass a, b;
    a.i = 100;      // access to i, j, and k is OK
    a.j = 4;
    a.k = a.i * a.j;
    b.k = 12;       // a.k and b.k are different
    cout << a.k << " " << b.k;
    return 0;
}
```

Question:
Explain the relationship between `struct` and `class` in C++. How are they similar and how are they different?

Answer:

Structures and Classes in C++

- C++ supports all the features of structures as defined in C.
- In C++, a structure can have both variables and functions as members.
- It can declare some of its members as 'private'.
- In C++, the structure names are stand-alone and can be used like any other type names.
- **student A; // C++ declaration**
- By default the members of a class are private, while, by default, the members of a structure are public.

- A class in C++ is syntactically similar to a `struct`.

- In C++, the role of `struct` has been expanded to act as an alternative way to specify a class.
- The only difference between a class and a struct is:
    - In a `struct`, members are public by default.
    - In a `class`, members are private by default.
- In all other respects, structures and classes are equivalent in C++.
- A `struct` can have variables and functions as members.
- In C++, a structure declaration defines a class type.

Justification for Having Both `struct` and `class`:

- There is no fundamental reason not to extend structures with more capabilities.
- In C, structures already grouped data; C++ builds on this by allowing them to include member functions.
- The similarity makes it easier to port existing C programs to C++.
- Having both `struct` and `class` provides flexibility:
    - `class` can evolve independently.
    - `struct` remains tied to C definition for compatibility.

Question:
What are unions in C++ and how are they used to define classes? Explain the concept of anonymous unions and list the restrictions applicable to both.

Answer:
Unions in C++:

- Like structures, unions can also be used to define a class in C++.
- A union may contain member functions, constructors, and destructors.
- Unions retain their C-like behavior, where all data elements share the same memory location.
- By default, union members are public, making them fully compatible with C.
- A union declaration defines a class type, thus preserving the principle of encapsulation.

Restrictions on Unions in C++:

- A union cannot inherit any class.
- A union cannot be a base class.
- Virtual member functions are not allowed.
- Static variables cannot be members of a union.
- A reference member cannot be used in a union.
- A union cannot contain any object that overloads the `=` operator.
- A union cannot contain any object that has an explicit constructor or destructor.

POD (Plain Old Data):

- The term POD applies to unions without member functions, constructors, or destructors.

Anonymous Unions:

- An anonymous union has no type name and no objects of it can be declared.
- Its member variables share the same location, just like a normal union.
- The member variables are accessed directly, without using the dot operator.
- The names of the members must not conflict with other identifiers in the same scope.

Additional Restrictions on Anonymous Unions:

- Only data elements are allowed.
- Member functions are not allowed.
- Cannot contain private or protected elements.
- If defined globally, they must be specified as static.

Question:
Differentiate between unions, classes, and structures in C++.

Answer:

Feature	Class	Structure	Union
Default Access Specifier	`private`	`public`	`public`
Inheritance Support	Yes	Yes	**No**
Virtual Functions	Allowed	Allowed	**Not allowed**
Static Members	Allowed	Allowed	**Not allowed**
Memory Allocation	Each member has its own location	Each member has its own location	**All members share same memory**
Object Containment	Can contain objects and pointers	Can contain objects and pointers	**Cannot contain objects with explicit constructors or destructors**

Use Case	For complete abstraction and OOP	For lightweight data structures with optional functions	For memory-efficient storage of variables when only one is used at a time

- Structs and classes are virtually equivalent, except for their default access specifiers.
- Unions differ significantly due to shared memory and stricter restrictions.

Question:
How is an object of a class created in C++? What happens during instantiation, and what are constructors?

Answer:

<u>Creating an Object of a Class:</u>

- Declaring a variable of class type creates an object.
- Example:
  - `Circle c;` → creates a regular object.
  - `Circle *c;` → creates a pointer to an object.

<u>Instantiation:</u>

- When an object is instantiated:
  - A new memory location is allocated to store its data members.
  - Code (member functions) is associated with the object via its class.
- You can create multiple objects from the same class type.

<u>Special Member Functions — Constructor:</u>

- A constructor is a public member function.
- It is automatically called when a new object is created.
- Used to initialize data members.
- Has the same name as the class.
- No return type (not even `void`).
- A class can have multiple constructors using function overloading.

<u>There are two ways to define member functions in C++:</u>

1. Member Functions Defined Outside the Class:

- Use the **binary scope resolution operator** `::`.
- Binds the function to its **class name**.
- Required when function code is **outside the class body**.

Syntax:

```
ReturnType ClassName::FunctionName() {
```

```
    // function code
}
```

Advantages:

- Helps organize large codebases.
- Allows different classes to have functions with the same name.

2. Member Functions Defined Inside the Class:

- Written directly inside the class body.
- No need to use scope resolution or class name.
- Functions defined inside the class are automatically inline.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class Circle {
private:
    double r;
public:
    Circle() { r = 0.0; }
    Circle(int val);
    void set(double val) { r = val; }
    double diameter() { return r * 2; }
    double area();
    double circumference();
};

Circle::Circle(int val) {
    r = val;
}

double Circle::area() {
    return r * r * (22.0 / 7);
}

double Circle::circumference() {
    return 2 * r * (22.0 / 7);
}
```

Question:
What is a friend function in C++? How is it declared, what are its uses, and what restrictions apply?

Answer:

A friend function is a non-member function that is given access to the private and protected members of a class.
It is declared inside the class using the `friend` keyword before its prototype.

Declaration Syntax:

Inside the class:
```
friend return_type function_name(arguments);
```

Usage:

- Even though it has access to private members, a friend function is **not a member** of the class.
- It is called like a **regular function**, **without** using the object name or the dot operator.
- It cannot be invoked as `object.function()`. Instead, it is called directly by name, e.g., `function_name(object);`

Advantages / Use Cases:

- Useful when overloading certain operators.
- Helpful in designing custom I/O functions.
- Allows efficient access when two or more classes contain interrelated members.
- Enables one function to access internal states of multiple objects efficiently.

Example Use Case (implied):

- A function needing to access private members of multiple classes simultaneously.

Restrictions:

- Friend functions are not inherited by derived classes.
- Friend functions cannot have storage-class specifiers like `static` or `extern`.

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

class Sample {
    int num;
public:
    void set(int n) { num = n; }
    friend int getDouble(Sample x);
};

// Friend function
int getDouble(Sample x) {
```

```
    return x.num * 2;
}

int main() {
    Sample obj;
    obj.set(5);
    cout << getDouble(obj);
    return 0;
}
OUTPUT:
10
```

Question:
How are class members accessed in C++? Explain the use of the dot and arrow operators. Include all relevant points.

Answer:

Class members are accessed using the same operators as in structures. These operators help invoke functions or access data inside class instances.

The two main operators used are:

1. Dot Member Selection Operator (`.`):

- Use `.` when working with objects or references.
- Syntax:
  ```
  object.member;
  reference.member;
  ```

<u>Example:</u>

```
Circle c;
c.setRadius(5.0);          // Accessing member function using
dot operator
double a = c.getArea();    // Accessing another member function
```

2. Arrow Member Selection Operator (`->`):

- Used when accessing a member through a pointer to an object.
- Syntax:
  ```
  pointer->member;
  ```

<u>Example:</u>

```
Circle* ptr = new Circle();
ptr->setRadius(7.0);             // Using arrow operator
double area = ptr->getArea();  // Accessing function via
pointer
delete ptr;
```

Additional Notes from the PDF:

- The dot (`.`) operator and arrow (`->`) operator are standard ways to access both member variables and member functions.
- These operators are identical in behavior to those used for structs.
- Whether using `.` or `->`, the class member must be accessible (e.g., `public`), or an error will occur.
- These are the only two operators used for direct access to class members in standard usage.

Read c++ unit 2 pdf from page 53-92

Question:
What is a friend class in C++? How is it declared, and what are its characteristics?

Answer:

A friend class is a class whose all member functions are allowed to access the private members of another class.
This access is granted using the `friend class` declaration inside the target class.

<u>Syntax:</u>

Written inside the class that is granting access:

```
friend class ClassName;
```

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class A {
    int x;
public:
    A(int val) { x = val; }
    friend class B; // B can access A's private members
};

class B {
public:
    void show(A obj) {
        cout << "Value of x is: " << obj.x << endl;
    }
};

int main() {
    A a(10);
```

```
    B b;
    b.show(a);
    return 0;
}
OUTPUT:
Value of x is: 10
```

Characteristics of Friend Classes:

- All member functions of a friend class can access private members of the class that declares it as a friend.
- Friendship is not mutual — if class A is a friend of class B, class B is not automatically a friend of class A.
- Friendship does not imply inheritance — friend class members do not become members of the class granting friendship.
- Used in special cases where tight coupling between two classes is needed.

Question:
What is an inline function in C++? How is it defined and used, and what are its advantages and limitations?

Answer:

An inline function is a short function whose code is expanded at the point of each call instead of being invoked through a regular function call.
It avoids function call overhead and is similar to a function-like macro.

Syntax:

Use the `inline` keyword before the function definition.

Example:

```
inline int max(int a, int b)

{

    return a > b ? a : b;

}
```

Example Program:

```
#include <iostream>
using namespace std;

inline int max(int a, int b) {
    return a > b ? a : b;
}
```

```
int main() {
    cout << max(10, 20);         // Outputs 20
    cout << " " << max(99, 88); // Outputs 99
    return 0;
}
```

Advantages:

- Increases speed by avoiding function call overhead.
- Ideal for very small, frequently used functions.
- Can be used for member functions of a class.

Limitations:

- May cause larger executable size due to code duplication.
- Compiler may ignore the `inline` request if the function is not suitable (e.g., too large or contains loops, recursion).
- Only small and performance-critical functions should be inlined.

Inline Class Member Functions Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

class MyClass {
    int a, b;
public:
    inline void setValues(int x, int y);  // inline declared
inside class
    inline void show();                   // inline declared
inside class
};

inline void MyClass::setValues(int x, int y) {
    a = x;
    b = y;
}

inline void MyClass::show() {
    cout << a << " " << b << endl;
}

int main() {
    MyClass obj;
    obj.setValues(10, 20);
    obj.show(); // Output: 10 20
    return 0;
```

```
}
OUTPUT:
10 20
```

Question:
How are inline functions defined within a class in C++? Provide an example and mention if constructors/destructors can be inlined.

Answer:

In C++, a function defined inside the class declaration is treated as an inline function.
It is not necessary to use the `inline` keyword — the compiler treats it as inline automatically.
However, using `inline` explicitly is also not an error.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class myclass {
    int a, b;
public:
    void init(int i, int j) { a = i; b = j; }          //
automatic inline
    void show() { cout << a << " " << b << "\n"; }    //
automatic inline
};

int main() {
    myclass x;
    x.init(10, 20);
    x.show();  // Output: 10 20
    return 0;
}
OUTPUT:
10 20
```

Constructor and destructor functions can also be inlined, either:

- Automatically if defined inside the class.
- Or explicitly by using the `inline` keyword.

Question:
What is a parameterized constructor in C++? How is it defined and what is its use? Provide an example.

Answer:

A parameterized constructor is a constructor that accepts arguments.
These arguments are used to initialize object data at the time of creation.
The parameters are added like in any regular function.
Initialization is done inside the constructor body using the arguments.

Advantages:

- Allows direct initialization of variables during object creation.
- Avoids the need for separate initialization function calls after creating an object.

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

class myclass {
    int a, b;
public:
    myclass(int i, int j) { a = i; b = j; } // parameterized
constructor
    void show() { cout << a << " " << b; }
};

int main() {
    myclass ob(3, 5); // object initialized using constructor
    ob.show();        // Output: 3 5
    return 0;
}
OUTPUT:
3 5
```

Question:
What is a static data member in C++? How is it declared, defined, and used? Explain with examples and output.

Answer:

A static data member is shared by all objects of a class.
Only one copy of the static variable exists for the class.
It is declared inside the class using the `static` keyword.
It must be defined outside the class using the scope resolution operator.
Static members are initialized to zero before any object is created.

Declaration and Definition Syntax:

```
class ClassName {
```

```
    static int var;  // Declaration inside class
};
int ClassName::var;   // Definition outside class (allocates
memory)
```

Example 1: Sharing Static Variable Across Objects

```
INPUT CODE:
#include <iostream>
using namespace std;

class Test {
    static int sharedValue; // shared by all objects
    int ownValue;           // unique to each object
public:
    void set(int s, int o) { sharedValue = s; ownValue = o; }
    void show();
};

int Test::sharedValue; // define static variable

void Test::show() {
    cout << "Shared value: " << sharedValue << endl;
    cout << "Own value: " << ownValue << endl;
}

int main() {
    Test obj1, obj2;

    obj1.set(5, 10);
    obj1.show();

    obj2.set(20, 30);
    obj2.show();

    obj1.show(); // sharedValue changed for all objects

    return 0;
}

OUTPUT:
Shared value: 5
Own value: 10
Shared value: 20
Own value: 30
Shared value: 20
Own value: 10
```

Accessing Static Variable Without Object:

Static members can be accessed independently using class name:
`ClassName::var;`

Example 2: Tracking Object Count Using Static Variable

```
INPUT CODE:
#include <iostream>
using namespace std;

class CountTracker {
public:
    static int total;
    CountTracker() { total++; }
    ~CountTracker() { total--; }
};

int CountTracker::total; // define static variable

void createTempObject();

int main() {
    CountTracker obj1;
    cout << "Objects alive: " << CountTracker::total << endl;

    CountTracker obj2;
    cout << "Objects alive: " << CountTracker::total << endl;

    createTempObject();

    cout << "Objects alive: " << CountTracker::total << endl;

    return 0;
}

void createTempObject() {
    CountTracker temp;
    cout << "Objects alive: " << CountTracker::total << endl;
    // temp gets destroyed after this function ends
}

OUTPUT:
Objects alive: 1
Objects alive: 2
Objects alive: 3
Objects alive: 2
```

<u>Key Points:</u>

- A static variable exists **before any object is created**.

- Shared across all objects of a class.
- Useful for:
    - **Access control** to shared resources.
    - **Tracking object instances**.
- Helps avoid global variables while preserving **encapsulation**.

Question:
What are static member functions in C++? What are their restrictions and how are they used? Explain with examples.

Answer:

A static member function belongs to the class, not to any specific object.
It can be called:
    - Using the class name and scope resolution operator, or
    - Using an object, though it doesn't use `this` pointer.

<u>Restrictions:</u>

- Can only access static members of the class.
- No `this` pointer inside static functions.
- Cannot have both static and non-static versions of the same function.
- Cannot be declared virtual, const, or volatile.

<u>Key Uses of Static Member Functions:</u>

- Preinitialize private static data before object creation.
- Provide class-level utility functions.
- Manage shared resources between all objects.

<u>Example 1: Controlling Access to Shared Resource:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class ResourceManager {
    static int resourceInUse; // shared by all objects
public:
    static int request();     // static function to request
resource
    void release() { resourceInUse = 0; }
};

int ResourceManager::resourceInUse; // define static variable

int ResourceManager::request() {
    if (resourceInUse)
```

```
        return 0; // already taken
    else {
        resourceInUse = 1;
        return 1; // granted
    }
}

int main() {
    ResourceManager obj1, obj2;

    if (ResourceManager::request())
        cout << "obj1 got the resource\n";
    if (!ResourceManager::request())
        cout << "obj2 was denied the resource\n";

    obj1.release();

    if (obj2.request())
        cout << "obj2 got the resource after release\n";

    return 0;
}
OUTPUT:
obj1 got the resource
obj2 was denied the resource
obj2 got the resource after release
```

Example 2: Initializing Static Data Before Object Creation:

```
INPUT CODE:

#include <iostream>

using namespace std;



class Demo {

    static int value; // shared by all objects

public:

    static void set(int x) { value = x; } // set static
variable

    void show() { cout << value << endl; }

};
```

```
int Demo::value; // define static variable


int main() {
    Demo::set(100); // set value before creating any object
    Demo obj;
    obj.show();     // Output: 100
    return 0;
}
OUTPUT:
100
```

Question:
When are constructors and destructors executed in C++? Explain with an example and output.

Answer:

Constructor Execution:

- A constructor is executed when the object is created.
- For a local object, the constructor runs when its declaration is encountered.
- For a global object, the constructor runs before `main()` starts.

Destructor Execution:

- A destructor is executed when the object is destroyed.
- Local destructors are called in the reverse order of construction when the block ends.
- Global destructors run in reverse order after `main()` terminates.

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

class Demo {
public:
    int id;
    Demo(int num);
    ~Demo();
};

Demo g1(1), g2(2); // global objects
```

```
Demo::Demo(int num) {
    id = num;
    cout << "Initializing " << id << endl;
}

Demo::~Demo() {
    cout << "Destructing " << id << endl;
}

int main() {
    Demo d1(3);
    cout << "This will not be first line displayed." << endl;
    Demo d2(4);
    return 0;
}
OUTPUT:
Initializing 1
Initializing 2
Initializing 3
This will not be first line displayed.
Initializing 4
Destructing 4
Destructing 3
Destructing 2
Destructing 1
```

Question:
What is the scope resolution operator `::` in C++? Explain its uses with an example.

Answer:

Used to link a class name with a member name to specify which class a member belongs to. Also used to access a global variable that is hidden by a local variable of the same name.

<u>Use Case 1 — Linking Class and Member:</u> Used to define or access class members outside the class.

```
ClassName::memberName
```

<u>Use Case 2 — Accessing Global Variable Hidden by Local:</u>

```
int i; // global variable

void f() {
    int i;       // local variable
    ::i = 10;    // sets global i to 10
}
```

- `::i` accesses the global `i` even though a local `i` is declared.

- Without `::`, `i = 10;` would refer to the local variable.

Key Point:

`::` accesses the enclosing/global scope when a name conflict occurs due to local declaration.

Question:
What are nested classes in C++? Where are they valid and how commonly are they used?

Answer:

- A nested class is a class defined inside another class.
- Since a class declaration defines a scope, the nested class is valid only within the scope of the enclosing class.
- Nested classes are rarely used in C++.
- The powerful inheritance mechanism in C++ usually eliminates the need for nesting classes.
- A nested class is scoped to its enclosing class and is generally not required in most designs due to better alternatives in C++.

Question:
What is a local class in C++? Where is it visible, what are its restrictions, and provide an example.

Answer:

A local class is a class defined inside a function.

It is known only within that function and cannot be used outside.

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

void f();

int main() {
    f();
    // myclass is not known here
    return 0;
}

void f() {
    class MyClass {
```

```
        int i;
    public:
        void put_i(int n) { i = n; }
        int get_i() { return i; }
    } obj;

    obj.put_i(10);
    cout << obj.get_i(); // Output: 10
}
OUTPUT:
10
```

Restrictions on Local Classes:

- All member functions must be defined inside the class.
- Cannot access local variables of the enclosing function.
    - Can only access static local or extern variables.
- Can access type names and enumerators from the enclosing function.
- Cannot declare static variables inside the local class.

Question:
How are objects passed to functions in C++? What happens with constructors and destructors during this process? Explain with an example and output.

Answer:

- Objects in C++ are passed to functions using call-by-value, like other variables.
- This means a copy of the object is made and passed to the function.
- The copy constructor is used to create the duplicate, and the destructor is called when the copy goes out of scope.

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

class MyClass {
    int i;
public:
    MyClass(int n);
    ~MyClass();
    void set_i(int n) { i = n; }
    int get_i() { return i; }
};

MyClass::MyClass(int n) {
    i = n;
    cout << "Constructing " << i << "\n";
```

```
}

MyClass::~MyClass() {
    cout << "Destroying " << i << "\n";
}

void func(MyClass obj); // function prototype

int main() {
    MyClass o(1);       // constructor called
    func(o);            // object passed by value (copied)
    cout << "This is i in main: " << o.get_i() << "\n";
    return 0;
}

void func(MyClass obj) {
    obj.set_i(2);
    cout << "This is local i: " << obj.get_i() << "\n";
}
OUTPUT:
Constructing 1
This is local i: 2
Destroying 2
This is i in main: 1
Destroying 1
```

Question:
How can a function return an object in C++? Explain the process with an example and mention what happens during object return.

Answer:

- A function in C++ can return an object just like any other data type.
- When an object is returned, a temporary object is created to hold the return value.
- After the value is returned, the temporary object is destroyed.
- This destruction can cause unexpected side effects if the object manages resources like dynamic memory.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

class Box {
    int value;
public:
    void set(int n) { value = n; }
    int get() { return value; }
```

```
};

Box makeBox() {
    Box temp;
    temp.set(10);
    return temp;
}

int main() {
    Box b;
    b = makeBox();
    cout << b.get() << endl; // Output: 10
    return 0;
}
OUTPUT:
10
```

Note:

- If the returned object uses dynamic memory, its destructor may free memory that is still in use by the receiving object.
- To avoid such issues, use a copy constructor and overload the assignment operator (topics covered later).

Question:
How does object assignment work in C++? Explain with an example and output.

Answer:

- In C++, if two objects are of the same class type, one object can be assigned to another using the = operator.
- This copies all data members from the object on the right side into the object on the left side.
- The copy is done using a bit-by-bit (shallow) copy by default.

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

class MyClass {
    int i;
public:
    void set_i(int n) { i = n; }
    int get_i() { return i; }
};
```

```
int main() {
    MyClass ob1, ob2;

    ob1.set_i(99);
    ob2 = ob1; // assign ob1 to ob2

    cout << "This is ob2's i: " << ob2.get_i(); // Output: 99
    return 0;
}
OUTPUT:
This is ob2's i: 99
```

Interface

Definition:

- In C++, an interface refers to the public part of a class that is accessible by the rest of the program.
- It includes public member functions and public variables (though best practice avoids public variables).
- The interface controls access to the internal (private) elements of an object.

Purpose:

- Provides a controlled access mechanism to interact with objects.
- Helps enforce encapsulation — internal details remain hidden.
- Promotes data protection and modular programming.

How it's Implemented:

- Through public member functions declared in a class.

```
class Example {
private:
  int data;  // private: cannot access directly

public:
  void setData(int d) { data = d; }      // public interface
  int getData() { return data; }         // public interface
};
```

- Here, `setData()` and `getData()` form the interface to access `data`.